



UNIVERSIDAD
NACIONAL DE SAN MARTÍN

GUÍA DE EJERCICIOS INTEGRADORES

Programación Avanzada

1 Ejercicios Integradores

1.1 Sistema de Gestión de Estudiantes

Crea una clase **Persona** con atributos **nombre** y **edad**. Implementa una clase hija **Estudiante** con atributos adicionales: **carrera**, **legajo** y **notas**. Agrega métodos para cargar una nota y calcular el promedio. Usa una comprensión de listas para obtener estudiantes con promedio mayor a 5. Guarda y lee los datos desde un archivo CSV, manejando excepciones si el archivo no existe.

1.2 Catálogo de Películas Online

Define una clase **Pelicula** con atributos **título**, **director**, **año** y **rating**. Crea una clase **Catalogo** que almacene películas en una lista. Agrega métodos para filtrar películas con **lambda + filter** (ej: `rating > 8`). Implementa un generador que devuelva películas año por año. Guarda y recupera el catálogo en un archivo JSON.

1.3 Sistema de Monitoreo Climático

Crea una clase **Sensor** con atributos **id**, **ubicación** y **tipo**. Crea una clase **Lectura** con atributos **fecha** y **valor**. Agrupa sensores y lecturas en una clase **EstacionClimatica**. Obtén datos reales con la librería **requests** (ej: API OpenWeather). Guarda los logs en un archivo de texto. Maneja excepciones si la API falla.

1.4 Tienda Online

Crea una clase **Producto** con atributos **código**, **nombre** y **precio**. Define una clase **Carrito** que almacene productos y calcule el total usando comprensiones de diccionarios. Implementa un decorador que registre cada compra en un archivo log. Utiliza polimorfismo con un método **calcular_descuento()** redefinido en productos especiales.

1.5 Sistema de Biblioteca Digital

Crea una clase **Libro** con atributos **título**, **autor** y **año**. Define una clase hija **LibroDigital** con atributos adicionales **URL de descarga** y **formato**. Agrega un método que cuente la cantidad de libros de un autor usando comprensiones. Implementa un generador que devuelva libros uno a uno al recorrer el catálogo. Guarda y carga los datos en un archivo binario con **pickle**.

1.6 Gestor de Finanzas Personales

Define una clase **Gasto** con atributos **fecha**, **categoría**, **monto**. Crea una clase **Ingreso** con atributos **fecha**, **fuentes**, **monto**. Agrupa todo en una clase **Balance** que calcule los totales. Agrega un método para ordenar gastos por

monto usando `lambda`. Implementa un decorador que valide que el monto no sea negativo. Guarda los movimientos en un archivo CSV y maneja excepciones de formato.

1.7 Plataforma de Cursos Online

Crea una clase `Curso` con atributos `nombre`, `profesor` y `alumnos`. Define una clase `Alumno` con atributos `nombre`, `email`, `notas`. Agrega un método que calcule el ranking de alumnos usando comprensiones y `sorted + lambda`. Simula el envío de resultados a un servidor con `requests` (POST). Maneja excepciones si el servidor no responde.

1.8 Sistema de Control de Vehículos

Crea una clase `Vehiculo` con atributos `marca`, `modelo`, `año`. Define subclases `Auto`, `Moto`, `Camion`. Usa polimorfismo implementando el método `calcular_peaje()`. Agrega un generador que devuelva vehículos por tipo. Implementa un decorador que registre accesos al sistema en un archivo log. Guarda todos los vehículos en un archivo JSON.

1.9 Gestor de Música

Crea una clase `Cancion` con atributos `título`, `artista`, `duración`. Define una clase `Playlist` que almacene canciones. Agrega un método que filtre canciones por duración usando `lambda`. Implementa un generador que devuelva canciones una a una para simular reproducción. Maneja excepciones si se intenta acceder a una canción inexistente. Guarda las playlists en un archivo de texto.

1.10 Sistema de Reservas de Hotel

Crea una clase `Habitacion` con atributos `número`, `tipo`, `precio`. Crea una clase `Reserva` con atributos `cliente`, `habitación`, `fecha`. Define una clase `Hotel` que gestione todas las reservas. Agrega un método que devuelva las habitaciones libres usando comprensiones de listas. Implementa un decorador que valide fechas antes de crear la reserva. Usa `requests` para consultar disponibilidad externa (API simulada). Maneja excepciones de red y registra en un archivo log.